

Enterprise Employee Analytics & Data Warehouse System

Problem Statement

Your company is migrating from legacy, disconnected systems to a centralized, cloud-ready Data Warehouse to track employee performance, attrition, and project allocations. Instead of a simple CLI and basic tables, you are tasked with building an end-to-end, Object-Oriented application that interfaces with a robust Data Warehouse.

HR and Executive teams will use a Streamlit Web Application to:

1. Onboard employees and manage complex project assignments.
2. Track historical changes in employee roles and salaries (SCD Type 2).
3. View advanced analytical dashboards driven by a Dimensional Data Model.

Technologies Involved

- Python 3: Advanced Object-Oriented Programming (OOP) for ETL pipelines, data synthesis, and app logic.
- MySQL Workbench: For both OLTP relational storage and OLAP Data Warehousing.
- Streamlit: For the frontend UI, data entry forms, and interactive dashboards.
- GitHub: For strict version control and collaboration.
- Cloud Computing: Streamlit Community Cloud (for deployment).
- Data Modeling Tools: Draw.io, Lucidchart, or similar for editable diagrams.

Deliverables

1. GitHub Repository: Containing all Python scripts, modularized using OOP principles.

2. Data Synthesizer Script: A Python script utilizing pandas and Faker to scale the base dataset to 100,000+ rows and generate historical SCD Type 2 records.
3. Architecture & Design Diagrams: Editable files for both the ER Diagram (OLTP) and Dimensional Model (OLAP).
4. Database Scripts: .sql files containing DDL, DML, Stored Procedures, CTEs, and Window Functions.
5. Streamlit Application: A fully functioning web app that runs locally or in the cloud.
6. Project Documentation: A comprehensive README.md and Google doc explaining the architecture, setup, and data warehousing logic.

Phase I: Architecture, Modeling, and Data Ingestion

1. Dataset Sourcing & Synthesizing (The Data Synthesizer Task):
 - Download the *IBM HR Analytics Employee Attrition & Performance dataset* (which contains approx. 1,470 rows).
 - Scale for Volume: Write a Python script using the pandas and Faker libraries to ingest this base data and synthetically scale it up to 100,000+ rows.
 - Engineer Historical Data: Because the IBM dataset is just a single snapshot in time, use Python to artificially generate historical records for a subset of employees (e.g., simulating that an employee changed departments, got promoted, or received a salary bump 2 years ago). You *must* engineer this data to satisfy the SCD Type 2 requirement.
 - Load this massive, synthesized dataset into a staging schema in MySQL Workbench.
2. OLTP Database Design (ER Diagram):
 - Design a highly normalized relational database for the application backend (Employees, Departments, Projects, Reviews, Assignments).

- Create an editable ER Diagram.
- 3. OLAP Data Warehouse Design (Dimensional Model):
 - Design a Star Schema containing:
 - Fact Table: Fact_PerformanceReviews (tracking scores, ratings).
 - Dimension Tables: Dim_Employee, Dim_Project, Dim_Date, Dim_Department.
 - Data Warehousing Concepts:
 - Implement Surrogate Keys for all Dimension tables.
 - Design Dim_Employee to support SCD Type 2 (Slowly Changing Dimensions) to track historical changes generated in your synthesizer script (using start_date, end_date, and is_current flags).
 - Create an editable Dimensional Model diagram.

Phase 2: Advanced SQL & ETL Development

1. Database Setup: Execute DDL commands to build your OLTP and OLAP schemas in MySQL.
2. ETL Pipelines (SQL & Python):
 - Write Stored Procedures to move data from the OLTP tables to the OLAP Data Warehouse.
 - Use CTEs and Window Functions in your SQL scripts to clean, rank, and transform the synthesized dataset before loading it into your Dimension and Fact tables.
 - Ensure your ETL logic correctly handles the SCD Type 2 inserts/updates when employee attributes change.

Phase 3: Python Application Logic (OOP)

Moving away from functional programming, build a modular Object-Oriented backend:

1. `db_manager.py`: Create a `DatabaseConnection` class utilizing Singleton design patterns to manage MySQL connections safely.
2. Entity Classes: Create classes for `Employee`, `Project`, and `Review` that encapsulate attributes and behaviors.
3. Data Access Layer (DAL): Create Manager classes (e.g., `EmployeeManager`, `AnalyticsManager`) that inherit from your DB classes to handle CRUD operations. Implement robust try-except blocks for error handling.

Phase 4: Streamlit UI and Interactive Dashboards

Replace the original command-line interface with a modern web application using Streamlit:

1. Data Entry Module (OLTP Interaction):
 - Build Streamlit forms to onboard new employees, create projects, and submit performance reviews.
 - Write logic to ensure that updating an employee's department via the UI triggers an SCD Type 2 update in the Data Warehouse.
2. Analytics Dashboard (OLAP Interaction):
 - Connect Streamlit directly to your Data Warehouse (Star Schema).
 - Generate interactive charts (using Plotly/Altair within Streamlit) showing:
 - Year-over-year performance trends.
 - Top-performing employees by department (utilizing SQL Window Functions like `DENSE_RANK()`).
 - Employee attrition risk or project bottleneck analysis.

Phase 5: Version Control & Cloud

1. Version Control:
 - All code must be pushed to GitHub.

- You must use feature branches (e.g., feature/database-design, feature/streamlit-dashboard) and merge them into main via Pull Requests.

2. Cloud Integration:

- Deploy your Streamlit application to the Streamlit Community Cloud so it can be accessed via a public URL.

Evaluation Criteria

- Data Synthesis & Scale: Ability to logically generate 100,000+ realistic rows and handle that volume without performance degradation.
- Data Warehousing: Correct implementation of Surrogate Keys, Star Schema, and SCD Type 2 logic using the synthesized historical data.
- SQL Mastery: Effective use of advanced SQL (CTEs, Window Functions, Procedures).
- Python Architecture: Clean, modular OOP design with proper encapsulation and error handling.
- Presentation: The usability, aesthetics, and analytical depth of the Streamlit dashboard.